Guía de actividades

2.3: Modelos lineales generalizados

Actividades conceptuales

1. Sea $\{y_1, \ldots, y_n\}$ una muestra i.i.d. de $Y \sim \text{Exponencial}(\mu)$, con $\mu > 0$. La función de densidad es

$$p(y; \mu) = \frac{1}{\mu} e^{-y/\mu}, \qquad y \geq 0,$$

con $\mathbb{E}[Y] = \mu$ y $\text{Var}[Y] = \mu^2$.

- **a.** Muestre que esta distribución pertenece a la familia exponencial e identifique el parámetro natural η, el estadístico suficiente $T(y)$ y la función de partición $A(\eta)$.
- **b.** Interprete Y como una variable de interés en un contexto aplicado (por ejemplo, el tiempo entre eventos). ¿Cómo calcularía la predicción puntual de Y a partir del MLE de μ?

2. A partir de la distribución Exponencial de la actividad anterior, formule el modelo lineal generalizado correspondiente.

- Indique la función de enlace canónica.
- Escriba la función de log-verosimilitud para una muestra.
- Obtenga el gradiente $\nabla \ell(\boldsymbol{\theta})$ y la Hessiana $\nabla^2 \ell(\boldsymbol{\theta})$ de la log-verosimilitud, usando las expresiones generales de la familia exponencial.

3. Escriba el algoritmo IRLS correspondiente al modelo de regresión Exponencial con enlace canónico. Indique explícitamente las expresiones de la media μ, la matriz de pesos W y la pseudo-respuesta z. Presente la fórmula de actualización de los parámetros. Describa además cómo se realiza la predicción para un nuevo dato $\mathbf{x}$.

4. Repita las actividades 1, 2 y 3 para la distribución de Poisson. Recuerde que si $Y \sim \text{Poisson}(\lambda)$, entonces $\mathbb{E}[Y] = \lambda$ y $\text{Var}[Y] = \lambda$.

Actividades computacionales

1. Implemente las clases `myIRLS` y `myGLMGradientDescent` que permitan ajustar modelos lineales generalizados mediante el algoritmo IRLS y mediante descenso por gradiente, respectivamente. Ambas clases deben recibir como parámetro la familia del modelo a utilizar y, a partir de esa elección, construir internamente μ, W y z (en el caso de IRLS) o el gradiente de la log-verosimilitud (en el caso de descenso por gradiente). La clase `myGLMGradientDescent` debe recibir además la tasa de aprendizaje y el número de iteraciones, y almacenar la evolución del costo a lo largo del entrenamiento.

2. Genere datos sintéticos a partir del modelo probabilístico correspondiente, tanto para regresión logística como para regresión de Poisson, y utilícelos para evaluar ambas clases implementadas.

a. Compare los coeficientes estimados por `myIRLS` y `myGLMGradientDescent` con los obtenidos mediante `GLM` de `statsmodels`, para ambas familias consideradas.

b. Compare el rendimiento de `myIRLS` contra `myGLMGradientDescent` en términos de convergencia (evolución del costo), tiempo de ejecución y calidad de las soluciones obtenidas.

3. Implemente la clase `mySoftmaxRegression` que permita aplicar regresión softmax en datos de clasificación multiclase mediante descenso por gradiente. La clase debe recibir como parámetros la tasa de aprendizaje y el número de iteraciones. Asegúrese que la clase:

- Considere el término de *bias* en los parámetros del modelo.
- Devuelva la predicción del modelo (probabilidades y etiqueta de clase) para nuevas muestras.
- Almacene la evolución del costo a lo largo de las iteraciones.

4. Utilice la clase `mySoftmaxRegression` para resolver un problema de clasificación sobre el conjunto de datos `load_wine` de `scikit-learn`.

> *El dataset* `load_wine` *contiene 178 muestras de vinos italianos provenientes de tres variedades distintas de uva. Cada muestra está descrita por 13 variables fisicoquímicas (alcohol, ácido málico, ceniza, alcalinidad de ceniza, magnesio, fenoles totales, flavonoides, fenoles no flavonoides, proantocianinas, intensidad de color, matiz, OD280/OD315 de vinos diluidos y prolina), y el objetivo es predecir la variedad de uva a partir de dichas variables. Por lo tanto, se trata de un problema de clasificación multiclase con tres clases de tamaño aproximadamente balanceado.*

a. Divida los datos en conjuntos de entrenamiento y prueba (*train-test*), estandarice las variables (ajustando el escalador únicamente con el conjunto de entrenamiento), entrene el modelo y reporte el *accuracy* obtenido en ambos conjuntos. Grafique la evolución del costo a lo largo de las iteraciones.

b. Compare los coeficientes estimados y las predicciones obtenidas con las de un modelo equivalente de `scikit-learn`, utilizando distintos algoritmos de optimización que empleen enfoques diferentes, y discuta si existen diferencias relevantes entre los coeficientes y entre las predicciones. Asegúrese de que ninguno de los modelos utilice regularización, de modo que todos optimicen la misma función de costo que `mySoftmaxRegression`.

c. Seleccione dos *features* que permitan una buena separación visual entre clases, entrene nuevamente el modelo utilizando únicamente dichas variables, y grafique las fronteras de decisión obtenidas.